Table of Contents

1. Scoped Action Enablement . . . 1
   1.1. Core principle . . . 1
   1.2. What the framework returns . . . 2
   1.3. Runtime models . . . 2
   1.4. REST API . . . 3
      1.4.1. Check request . . . 3
      1.4.2. Check response . . . 4
      1.4.3. Manifest response . . . 5
   1.5. Manifest model and contributor SPI . . . 5
      1.5.1. Contributor precedence . . . 5
   1.6. Resolver SPI . . . 6
   1.7. Built-in OSS dependency types . . . 6
   1.8. Evaluation behavior . . . 7
   1.9. Example contributor . . . 7
   1.10. Relationship to permissions and UI actions . . . 8
   1.11. Enterprise extensions . . . 9
   1.12. Guidance . . . 9

Chapter 1. Scoped Action Enablement

Scoped action enablement is the framework-level answer to the question:

"Why is this action unavailable right now?"

Permission rules already answer:

"May this identity perform `area / functionalDomain / action`?"

Action enablement adds the missing operational layer so applications, shells, and setup dashboards can also answer:

- is the action allowed by policy?
- is the action enabled for this rollout or cohort?
- is the action operationally ready, or is some setup still missing?

This guide documents the runtime API, manifest model, SPI contracts, and the framework's built-in dependency types.

1.1. Core principle

The atomic unit is still the same scoped action tuple already used by Quantum security:

```
area / functionalDomain / action
```

Examples:

- `integration / exchange / create`
- `integration / guided_integration / create`
- `integration / workflow / create`
- `system / action-enablement / check`

Quantum intentionally does not introduce a second authorization abstraction. The same tuple defined by `@FunctionalMapping` and `@FunctionalAction` remains canonical for:

- permission checks
- UI action derivation
- action enablement manifests
- operational blocker reporting

Capabilities are still useful, but they are an application-level grouping over one or more scoped actions, not a replacement for the scoped action tuple itself.

1.2. What the framework returns

Each evaluated action produces four booleans:

Field	Meaning
`allowed`	The permission rule stack allows the action for the evaluated identity and data domain
`enabled`	Rollout and administrative toggles, especially feature flags, say the action should be exposed
`ready`	Operational prerequisites such as settings, referenced entities, or external runtime dependencies are satisfied
`usable`	Convenience result equal to `allowed && enabled && ready`

This separation is important because it points the user to the right remediation path:

- `allowed=false` usually means a policy or role issue
- `enabled=false` usually means a feature flag or rollout issue
- `ready=false` usually means configuration, provisioning, or runtime setup is incomplete

1.3. Runtime models

The runtime contracts live in `quantum-action-enablement-models`.

```
public class ScopedActionRef {
    String area;
    String functionalDomain;
    String action;
}

public class DependencyCheckRef {
    String type;
    String refName;
    Map<String, Object> config;
}

public class ScopedActionRequirement {
    ScopedActionRef scopedAction;
    String displayName;
    String description;
    List<DependencyCheckRef> dependencies;
}

public class ScopedActionEnablementStatus {
    ScopedActionRef scopedAction;
    boolean allowed;
    boolean enabled;
```

```
    boolean ready;
    boolean usable;
    List<EnablementBlocker> blockers;
}
```

Each blocker is machine-readable and user-displayable:

```
public class EnablementBlocker {
    EnablementImpact impact;  // ALLOWED, ENABLED, READY
    String type;              // permission, feature-flag, setting-present, ...
    String code;              // permission-denied, setting-missing, ...
    String message;           // human-readable explanation
    String severity;          // warn or error
    Map<String, Object> metadata;
}
```

1.4. REST API

The runtime API lives in `quantum-action-enablement-quarkus` and is exposed by `ActionEnablementResource`.

Endpoints:

- `POST /system/actions/enablement/check`
- `GET /system/actions/enablement/manifest`

The resource itself is mapped as:

- `area=system`
- `functionalDomain=action-enablement`
- `action=check|view`

1.4.1. Check request

```
{
  "identity": "tenant-admin",
  "realm": "acme",
  "roles": ["admin"],
  "orgRefName": "acme-org",
  "accountNumber": "1000000001",
  "tenantId": "acme-tenant",
  "dataSegment": 0,
  "ownerId": "tenant-admin",
  "scope": "api",
  "actions": [
    {
      "area": "integration",
```

```
      "functionalDomain": "guided_integration",
      "action": "create"
    },
    {
      "area": "integration",
      "functionalDomain": "workflow",
      "action": "create"
    }
  ]
}
```

1.4.2. Check response

```
{
  "results": [
    {
      "scopedAction": {
        "area": "integration",
        "functionalDomain": "guided_integration",
        "action": "create"
      },
      "allowed": true,
      "enabled": false,
      "ready": false,
      "usable": false,
      "blockers": [
        {
          "impact": "ENABLED",
          "type": "feature-flag",
          "code": "feature-flag-disabled",
          "message": "Feature flag 'guided-integrations' is disabled.",
          "severity": "warn"
        },
        {
          "impact": "READY",
          "type": "setting-present",
          "code": "setting-missing",
          "message": "Configuration setting 'quantum.integration.shared-storage.root'
is not set.",
          "severity": "error"
        }
      ]
    }
  ]
}
```

1.4.3. Manifest response

`GET /system/actions/enablement/manifest` returns the registered `ScopedActionRequirement` entries after all CDI contributors have been aggregated.

That endpoint is useful for:

- setup dashboards
- shell tools
- test fixtures
- verifying what requirements are active for a given application build

1.5. Manifest model and contributor SPI

The dependency relationships between settings, feature flags, and scoped actions are expressed as `ScopedActionRequirement` entries.

Applications contribute requirements through CDI:

```
public interface ScopedActionRequirementContributor {

    default int priority() {
        return 100;
    }

    Collection<ScopedActionRequirement> requirements();
}
```

The framework aggregates contributors through `ScopedActionRequirementRegistry`:

```
public interface ScopedActionRequirementRegistry {

    Optional<ScopedActionRequirement> find(ScopedActionRef ref);

    List<ScopedActionRequirement> list();
}
```

This is where you encode relationships such as:

- `integration / guided_integration / create` depends on feature flag `guided-integrations`
- `integration / exchange / create` depends on tenant filesystem provisioning
- `system / export / view` depends on a specific config property being present

1.5.1. Contributor precedence

Contributors are sorted by ascending `priority()`. Requirements are then indexed by scoped action

key.

Practical implication:

- later contributors for the same scoped action replace earlier entries
- the framework's own contributor can provide a default manifest entry
- applications can override or specialize that entry with their own contributor

1.6. Resolver SPI

Each dependency type is resolved by an `ActionDependencyResolver`:

```
public interface ActionDependencyResolver {

    String supportsType();

    DependencyResolutionResult evaluate(
        DependencyCheckRef dependency,
        EnablementEvaluationContext context
    );
}
```

`EnablementEvaluationContext` carries the runtime inputs:

- `identity`
- `realm`
- `roles`
- `scope`
- `DataDomain`
- target `ScopedActionRef`

This gives each resolver enough context to evaluate readiness in the correct tenant, realm, and user scope.

1.7. Built-in OSS dependency types

The open-source framework currently ships with these resolver types:

Dependency type	Meaning	Impact
`permission`	Run the existing permission engine for the scoped action	`ALLOWED`
`feature-flag`	Require that a `FeatureFlag` exists and is enabled in the active realm	`ENABLED`
`setting-present`	Require that a named configuration property exists and is non-blank	`READY`

Dependency type	Meaning	Impact
`entity-exists`	Require that a referenced entity exists in the current realm	`READY`

Representative blocker codes include:

- `permission-denied`
- `feature-flag-ref-missing`
- `feature-flag-missing`
- `feature-flag-disabled`
- `setting-name-missing`
- `setting-missing`
- `entity-dependency-invalid`
- `entity-model-unresolved`
- `entity-missing`

1.8. Evaluation behavior

`ScopedActionEnablementService` evaluates dependencies for each requested action and aggregates blockers by impact.

At a high level:

1. Resolve the manifest entry for the scoped action.
2. Build an `EnablementEvaluationContext` from the request plus current security context defaults.
3. Resolve each dependency by type.
4. Collect blockers from all failing dependencies.
5. Derive `allowed`, `enabled`, `ready`, and `usable`.

Important framework behavior:

- If no manifest entry exists, the service creates a synthetic fallback requirement with only a `permission` dependency.
- The response also includes a `manifest-missing` blocker with `impact=READY`.
- If a dependency type has no registered resolver, the response includes `unsupported-dependency-type`.

This means action enablement is designed to fail visibly, not silently.

1.9. Example contributor

The framework itself contributes manifest entries for the action-enablement API. Applications then contribute their own operational requirements.

```java
@ApplicationScoped
public class MyActionEnablementManifestContributor
        implements ScopedActionRequirementContributor {

    @Override
    public Collection<ScopedActionRequirement> requirements() {
        return List.of(
            ScopedActionRequirement.builder()
                .scopedAction(ScopedActionRef.builder()
                    .area("integration")
                    .functionalDomain("guided_integration")
                    .action("create")
                    .build())
                .displayName("Generate Guided Integration")
                .description("Requires policy, rollout enablement, and baseline
setup.")
                .dependencies(List.of(
                    DependencyCheckRef.builder().type("permission").build(),
                    DependencyCheckRef.builder()
                        .type("feature-flag")
                        .refName("guided-integrations")
                        .build(),
                    DependencyCheckRef.builder()
                        .type("setting-present")
                        .refName("quantum.integration.shared-storage.root")
                        .build()
                ))
                .build()
        );
    }
}
```

This is the practical link between settings and capabilities:

- the capability is what the product or UI wants to describe
- the scoped action is the framework key
- the manifest records which settings, feature flags, and runtime dependencies attach to that scoped action

1.10. Relationship to permissions and UI actions

Action enablement complements existing security behavior rather than replacing it.

- `@FunctionalMapping` and `@FunctionalAction` define the scoped action tuple.
- Permission rules decide whether the action is authorized.
- `fillUIActions()` still reports which actions are allowed from the permission perspective.
- Action enablement explains why the action should still be disabled or annotated because rollout

or readiness conditions are not met.

This makes it especially useful for:

- onboarding flows
- setup dashboards
- admin consoles
- explainable disabled buttons
- CLI and MCP tooling

1.11. Enterprise extensions

The framework module is intentionally generic. Enterprise modules can contribute deeper operational resolvers through the same `ActionDependencyResolver` SPI.

Current enterprise resolvers include checks such as:

- `managed-secret-configured`
- `tenant-filesystem-provisioned`
- `standard-mounts-present`
- `workflow-runtime-reachable`

Those enterprise resolvers answer questions like:

- has the tenant filesystem been provisioned?
- do required standard mounts exist?
- is the managed secret configured?
- can the workflow runtime be reached?

The framework documentation stops at the extension point and shared response model. Enterprise-specific operational semantics should be documented in the enterprise module.

1.12. Guidance

- Keep authorization canonical in permission rules. Do not re-implement policy logic inside custom readiness resolvers.
- Use `feature-flag` for rollout state, not for durable authorization.
- Use `setting-present` and `entity-exists` for generic readiness checks.
- Put domain-heavy infrastructure checks in application or enterprise resolvers.
- Keep blocker codes stable because UIs and automation may key off them.
- Treat capabilities as UX groupings over scoped actions, not as a replacement for `area / functionalDomain / action`.